Small Yet Efficient: Fine-Tuning Qwen 3 for STEM MCQA Mastery

Aidas Venckunas | SCIPER 325464 | aidas.venckunas@epfl.ch
Quentin Angéloz | SCIPER 325935 | quentin.angeloz@epfl.ch
Yasmine Tligui | SCIPER 341215 | yasmine.tligui@epfl.ch
Jiwon You | SCIPER 394716 | jiwon.you@epfl.ch
Word2Wreck

Abstract

This project explores post-training strategies to specialize the Qwen3-0.6B-Base model as an efficient AI tutor for STEM tasks. We evaluate four methods: Supervised Fine-Tuning (SFT) for reasoning, Direct Preference Optimization (DPO) for alignment, Retrieval-Augmented Generation (RAG) for factual grounding, and quantization for efficiency. Key contributions include a mixed-reasoning dataset, the application of DPO, and an assessment of RAG and quantization.

1 Introduction

The rise of LLMs has enabled AI-powered educational tools, particularly specialized STEM tutors. While large models excel in reasoning, their high computational cost and limited customizability hinder accessibility. This has led to interest in fine-tuning smaller, open-source models for expert tasks, as demonstrated by T5's success in transfer learning for question answering[16].

Modern approaches favor efficient methods like Parameter-Efficient Fine-Tuning (PEFT) with LoRA[14], which adapts models without the cost of full fine-tuning. However, creating a robust AI tutor requires addressing reasoning, alignment, factuality, and efficiency. This project compares four post-training methodologies on the compact Qwen3-0.6B-Base model to tackle these challenges:

1. Supervised Fine-Tuning (SFT): We firstly construct a Science MCQA focused dataset and fine-tune an MCQA model directly on it. Later on, we build on work like Tülu 3[15] by developing a novel **mixed-reasoning dataset** that combines direct answers with Chain-of-Thought style reasoning in an aim to improve the model's core inferential abilities.

2. Direct Preference Optimization (DPO): To align model outputs with human judgments, we apply DPO[24], a stable and effective alternative to traditional reinforcement learning from human feedback (RLHF).

3. Retrieval-Augmented Generation (RAG): To refine factual accuracy, we implement a RAG pipeline based on the foundational framework[19], grounding the model in an external knowledge corpus.

4. Quantization: To improve deployment efficiency, we apply post-training quantization, a common technique for reducing model size and latency.

By evaluating each technique, we highlight the trade-offs and demonstrate the value of a multi-faceted approach to building efficient AI tutors from small models, despite the challenges of STEM MCQA tasks.

2 Approach

2.1 DPO Model

We fine-tuned the Qwen3-0.6B-Base model using DPO to improve response generation for scientific prompts. Initially, we applied full SFT with Hugging Face's AutoModelForCausalLM, an autoregressive model that generates token outputs, conditioned on previous tokens, to generate responses and align them with the expected ones, to adapt the model to scientific domains. For preference alignment, we used DPOTrainer, which trains the model on ranked pairs of preferred and rejected responses, the objective being to increase the likelihood of the preferred response and decrease the rejected one, based in our case on sigmoid preference loss function. We employed LORA[14] with two configurations of trainable parameters to enhance training efficiency, and tried two datasets to see which one enhanced generalization on the test set.

2.2 MCQA (SFT) Model

The primary goal for the MCQA model was to maximize its accuracy on STEM-based multiple-choice questions. Our approach evolved from a standard fine-tuning procedure to a more sophisticated strategy involving dataset augmentation. Initially, we planned to add a classification head to the base model to predict the correct answer identifier. However, due to implementation complexities with the `lighteval` evaluation harness, we pivoted to a generative approach. We pursued two parallel fine-tuning strategies:

1. Full Supervised Fine-Tuning (SFT): Updating all parameters of the Qwen3-0.6B-Base model.

2. Parameter-Efficient Fine-Tuning (PEFT): Using Low-Rank Adaptation (LoRA)[14] to update a small subset of parameters, reducing computational requirements and mitigating catastrophic forgetting.

Both methods were optimized using a standard causal language modeling loss corresponding to the correct answer. Initial training featured fine-tuning on simple question-answer choice pairs collected from science-related open-source MCQA datasets, with a total of 59,277 entries. The next step was the creation of a **mixed-reasoning dataset**. Inspired by recent work like Tülu 3[15], we hypothesized that teaching the model to generate step-by-step reasoning before the final answer would improve its problem-solving abilities. To achieve this, we augmented our training data with Chain-of-Thought (CoT) reasoning traces. These traces were generated by a teacher model (Qwen3-235B-A22B) for a portion of the dataset. Additionally, we incorporated expert-written reasoning already present in the MathQA dataset[8]. This resulted in a final training corpus where 45% of the examples included a reasoning trace before the answer, while the remaining 55% were direct question-answer pairs to maintain the model's ability to provide concise answers in the form of answer choice.

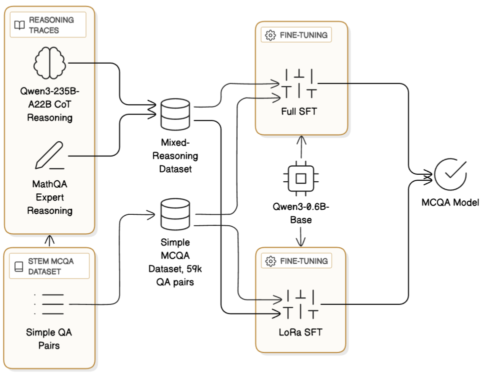

Figure 1: Dual MCQA supervised fine-tuning pipeline using simple MCQA and reasoning augmented datasets.

2.3 Quantized Model

We explored several one-shot and calibration-based quantization pipelines, the ones using static quantization all used a 512-sample calibration set from validation. We first explored the LLM-compressor library. We applied SmoothQuant and GPTQ for 4-bit and 8-bit activations (static and dynamic) quantization. However, we encountered problems loading these quantized models with VLLM on lighteval and measuring the used VRAM also was non-trivial. To ensure that the model could run, we settled on weight-only quantization using BitsAndBytes, testing standard 8-bit[9], CPU-offloaded and outlier-free 8-bit (threshold=0.0), skip-module (`lm_head` and `q_proj`) and embedding-quantized 8-bit, 4-bit NF4[10] both with and without double quantization (and with embedding quantization), as well as pure 4-bit FP4 (with thresholding). This provided good results with a minimal loss in performance.

2.4 RAG Model

Our RAG system is composed of two components to augment the MCQA model: (1) a document corpus, (2) a bi-encoder.

Encoder Fine-tuning: In our retrieval system, both queries and documents are embedded using the same encoder. We retrieve documents with high cosine similarity in this shared embedding space.

To improve retrieval quality, we fine-tuned the encoder on question–answer pairs using contrastive learning. This is well aligned with the purpose of retrieval, since any ground truth answer is indeed relevant to the question.

Positive Pairs — A question and its corresponding answer or rationale from the same QA example.

Negative Pairs — A question and a randomly sampled answer from a different example.

We finetuned the encoder using a cosine similarity loss, encouraging high similarity for positive pairs and low similarity for negative pairs. Examples of such pairs are given in Table 8.

2.5 Data

2.5.1 DPO Model Data

For the SFT dataset, we used science datasets: Math-reasoning-10K[6], CodeParrot[11], MedQuad[7], Physics-Alpaca[13], and SciQ[4]. Using an LLM-based filtering approach, we prompted GPT to assess the relevance of each question–answer pair. The model returned a relevance score, and we retained only the top-ranked examples of each dataset to ensure good quality of samples. After cleaning, the resulting dataset contained 40,481 SFT samples (90% training, 10% validation).

For the initial DPO dataset, we used three preference datasets: Code-Preference-Pairs[18], Math-Step-DPO-10K[2], and 20,000 samples from the Ultrafeedback Binarized Preferences dataset[5]. Although not entirely scientific, these sources helped the model learn preference patterns. The combined dataset included 40,742 samples (90% train, 10% validation). For evaluation, we used a test set of 1937 science-related prompts extracted from the Intel Orca DPO Pairs dataset[1], filtered again using an LLM.

To further improve performance, we extended the training set by adding 9,000 scientific examples from Ultrafeedback[5] and 24,431 from the Multifaceted-Collection-DPO dataset[3], also filtered via LLM. This resulted in a dataset of 74,312 samples (95% training, 5% validation), while keeping the test set unchanged.

2.5.2 MCQA Model Data

The foundation of our MCQA training was a composite dataset aggregated from six public benchmarks: **CommonsenseQA**[25], **OpenBookQA**[21], **QASC**[17], **SciQ**[26], **MathQA**[8], and the text-only portion of **ScienceQA**[20]. The dataset was also complemented by all multiple choice questions given in the course provided preference pair dataset. After de-duplication, this resulted in a training set of 59,277 questions and a diverse validation set of 9,266 questions. This dataset was then also transformed into a mixed-reasoning dataset. We processed the 59,277 training questions to create two formats:

Direct QA: 32,417 questions (54.7%) formatted as standard prompt-answer pairs.

Reasoning QA: 26,860 questions (45.3%) where the answer is preceded by a CoT reasoning trace that was either generated by the teacher model or presented in the original dataset (**MathQA**) as expert reasoning feature.

Both the standard (original) and mixed datasets were used in different experiments to train our final SFT and LoRA models.

2.5.3 Quantized Model Data

All quantized variants were derived from the same fine-tuned MCQA model and thus share its original training data. No additional training or data augmentation was performed for the bitsandbytes post-training quantization. For static quantization experiments with the compressed-tensors library, we used 512 examples drawn from the validation split solely to calibrate activation ranges.

2.5.4 RAG Model Data

We constructed a unified dataset that serves as both the document corpus and the training data for the encoder. It merges Wikipedia STEM content[12] and ground-truth answers from STEM QA pairs(MedQuad[7], Physics-Alpaca[13]). An auxiliary `question` field contains the corresponding question, if the document originates from a QA pair. Collectively, this makes 40k QA pairs and 26k documents from Wiki dumps.

2.6 Evaluation Method

All generative models (MCQA, RAG, Quantized) were evaluated on their accuracy on the held-out validation set of 9,266 MCQA questions. We used the `lighteval` framework, an implementation of the EleutherAI Harness, to ensure standardized and reproducible evaluation, also for DPO evaluation.

2.7 Baselines

Our primary baseline for all generative models was the zero-shot performance of the **Qwen3-0.6B-Base** model on our validation set. This allows us to quantify the improvements gained from our various post-training procedures. Each augmented model (Quantized, RAG) is also compared against its non-augmented counterpart (the fine-tuned MCQA model).

2.8 Experimental Details

2.8.1 DPO Model

First approach : SFT

Experimental details: To incorporate full SFT to improve Qwen3-0.6B-Base's ability to respond to scientific questions, we used the AutoModelForCausalLM from Hugging Face to generate responses. To follow the model's performance and improvement, we use Cross-Entropy Loss, which is automatically computed by the model: $\mathcal{L} = -\sum_{i=1}^{N} \sum_{c=1}^{C} y_{ic} \log(p_{ic})$.

Evaluation method: We compare models performance using ROUGE-L, assessing the semantic similarity of the generated responses to the validation set responses.

Second approach : DPO to fine-tune Qwen

Experimental details: We used Qwen3-0.6B-Base as both the training model and reference for DPO. Because of OOM, we quantized the model's weights to 4 bits and used LoRA. We tested two scenarios: r=32, alpha=64 (1.21% trainable parameters), and r=320, alpha=640 (21.7% trainable parameters). In the 1.21% case, LoRA was applied to attention layers (query, value, and output projections), as these layers are crucial for DPO since they control how the model processes relationships between tokens, influencing the ability to capture context in chosen and rejected responses. For the 21.7% case, we added two trainable MNL layers (up_proj and down_proj). We used sigmoid loss for preference-based optimization, distinguishing between preferred and non-preferred answers: $\mathcal{L} = -\mathbb{E}\big[\log\sigma\big(\beta(\log\frac{\pi\theta(y_+|x)}{\pi_{\text{ref}}(y_+|x)} - \log\frac{\pi_\theta(y_-|x)}{\pi_{\text{ref}}(y_-|x)})\big)\big]$. The parameters used in all approaches are mentioned here : A.2)

Evaluation method: To compare the results obtained, we are going to use the lighteval accuracy, that evaluates the DPO model by comparing the differences in log-probabilities between the fine-tuned model and the qwen reference model, so for Qwen base model, we have an accuracy of 0 and we consider the new model to have improved when the lighteval accuracy is greater than 0.5, meaning it satisfies $\Delta_{\text{chosen}} > \Delta_{\text{rejected}}$, while a score below 0.5 indicates that the Qwen model performs better.

Third approach : New dataset

We used the same loss and evaluation metrics as in the second part, with DPOTrainer.

Testing different learning rates and beta values (which control the sharpness of preference alignment in the loss) could have provided further insights, but given the platform issues, we were limited in the number of runs we could perform.

2.8.2 MCQA Model

Based on initial experiments, we identified optimal hyperparameters for our two main training strategies.

As for Full SFT, the model was trained for 2 epochs with the AdamW optimizer, a learning rate of 5×10^{-5}, and a cosine learning rate schedule. Further experiments with lower learning rates (2×10^{-6} and 2×10^{-7}) did not yield improvements, resulting in a lower accuracy and confirming our hyperparameter choice.

For the LoRa PEFT, we trained for 2 epochs with a learning rate of 2×10^{-4}. We used a LoRA rank (r) of 8 and an alpha of 32.

These configurations were used to train models on both the standard (direct QA only) and the final mixed-reasoning datasets.

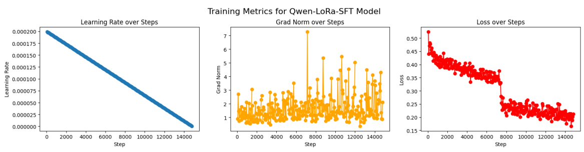

Figure 2: Training curve of LoRa PEFT model training run on a non-reasoning MCQA dataset.

2.9 Results

2.9.1 DPO Model Results

First approach : SFT

Results with Rouge-L metric on validation set: Qwen score : 0.198 and Qwen+SFT score : 0.174

Figure 3: SFT training loss over steps

Analysis: The addition of SFT showed some learning progress, as the training loss decreased over time. However, despite this improvement, the SFT performance score did not surpass the original Qwen score. This suggests that even if the model is able to learn from data, these gains did not translate in the general response quality. This indicates that the dataset used for fine-tuning may not have been optimal or that Qwen already performs well on scientific questions.

Second approach : DPO to fine-tune Qwen

Results:

Model DPO	Validation	Test	mnlp_dpo_eval[22]
Lora 1,21%	0.8172	0.5379	0.8220
Lora 21,7%	0.8292	0.5942	0.7633

Table 1: Accuracy on STEM DPO eval datasets across different Lora % of trainable parameters

Analysis: The results show that the 21.7% Lora model slightly outperforms the 1.21% model. However, both configurations generalize poorly on a 100% unseen dataset, that could be attributed to overfitting, where the model learned too much from the training data, or to the lack of diversity in the dataset. But the model performs well on validation sets, so the issue stems from the insufficient variability in the dataset, a direction that will be explored in the following section.

Third approach : New dataset
Results: Analysis: Before, the model's generalization on a fully unseen test dataset was not ideal but after completing the dataset, the model's performance significantly improved ($\approx$ 14% better on testing), highlighting the critical importance of dataset quality and diversity.

Model DPO	Validation	Test	mnlp_dpo_eval[22]
Lora 21,7%	0.7169	0.6815	0.7879

Table 2: Accuracy on STEM DPO eval for the new dataset

2.9.2 MCQA Model Results

After finding optimal training hyperparameters and evaluating the baseline model on our gathered validation set, we conducted an ablation study to measure the impact of our different training strategies and datasets.

Configuration	Training Data	Validation Acc.
Qwen3-0.6B-Base	–	0.334
Qwen-Full-SFT	Standard	0.515
Qwen-LoRA-SFT	**Standard**	**0.589**
Qwen-Full-SFT	Mixed Reasoning	0.477
Qwen-LoRA-SFT	Mixed Reasoning	0.528

Table 3: Ablation study of MCQA model fine-tuning performance.

The results clearly demonstrate the effectiveness of our approach, although also reveal surprising insights. Firstly, both full fine-tuning and LoRA significantly outperform the zero-shot baseline. Across both datasets, LoRA consistently achieves higher accuracy than full fine-tuning, suggesting that its regularization properties are beneficial for this task, while also underlying the parameter weight sensitivity problem in small LLM models even with low learning rates. This hypothesis is further reinforced with consistent high gradient norm values throughout the full fine-tuning, fluctuating between values of 20-30.

The most surprising finding is that the introduction of our mixed-reasoning dataset did not provide performance boost. This could be motivated by an imbalance of reasoning vs. non-reasoning samples in the mixed dataset. In order to promote model's reasoning capabilities while retaining the ability to answer briefly in needed contexts, the related works, such as Tülu 3[15] suggesting using higher reasoning ratios, e.g. 75% over 25% reasoning versus non-reasoning split.

Nonetheless, the final best performing LoRA model fine-tuned on a standard non-reasoning dataset improved upon the baseline model by a margin of 0.253, reaching a final accuracy of 58.9%.

2.9.3 Quantized Model Results

We evaluated a variety of BitsAndBytes quantization configurations using a fixed batch size of 128. For each variant, accuracy was measured using lighteval metrics, and peak GPU memory usage was recorded. We then computed a performance score as the accuracy percentage divided by the peak VRAM usage in gigabytes. The 4-bit NF4 variant with nested double quantization achieved the highest score and was chosen as the final model. It reduces the size of the model by a factor of 2.2 while losing only 2% accuracy on our test set, and reducing Peak VRAM usage by a factor 3.44.
In general quantization reduced significantly the size of the model, while losing only 0.2% accuracy for 8-bit quantization and 2-4% accuracy for 4-bit quantization. For more extensive data about model comparisons, please refer to the appendix.

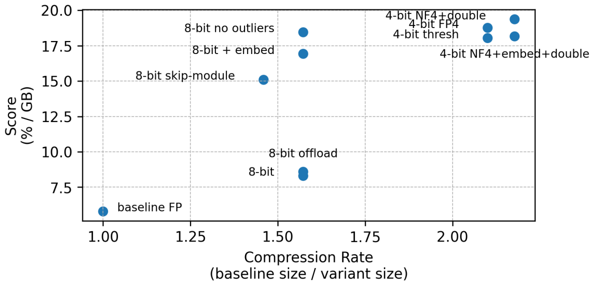

Figure 4: Performance score (%) vs. compression rate.

2.9.4 RAG Model Results

Validated on `MNLP_STEM_mcqa_evals`[23] dataset, we report accuracy across four configurations: **MCQA only** (baseline); **RAG with Base Encoder**(`bge-small`); **RAG with Finetuning**, base encoder finetuned on QA pairs; and **RAG (chunked corpus)**, finetuned encoder with the document corpus chunked to 64 tokens. For all RAG setups, we retrieve **top-1** document.

Configuration	Accuracy
MCQA Only	**0.3351**
RAG with Base Encoder	0.2831
RAG with Finetuned Encoder	0.2870
RAG with chunked corpus	**0.2922**

Table 4: Accuracy on STEM MCQA evaluation dataset across different RAG configurations.

Surprisingly, the addition of RAG did not surpass baseline MCQA model. Finetuning the encoder and chunking document corpus produced slight improvements each from RAG with Base encoder.

We suspect that attaching long documents to the end of a question can distract the MCQA model, especially if they are irrelevant. For RAG to yield real benefits, the queries should involve niche information that MCQA model doesn't know from pretraining.

Retrieval Evaluation:
We also evaluate the impact of encoder finetuning on retrieval. For each query, a retrieval is considered correct if the retrieved document is the corresponding answer to the question. In Table 5, encoder finetuning improved retrieval quality by 7%p.

Model	Top-1 Retrieval Accuracy
Base Encoder (`bge-small`)	61%
Finetuned	68%

Table 5: Top-1 retrieval accuracy of encoders evaluated on test split of QA pairs from RAG dataset

3 Ethical Considerations

When developing an AI tutor, it is crucial to consider the broader impact.

If the model works as intended, STEM students benefit from a free, accessible learning aid. However, harms could arise if the model provides consistently incorrect or misleading information, potentially hindering learning. Over-reliance on the tool could also discourage students from developing their own problem-solving skills.

The model was trained on English-language STEM datasets, which may not reflect the curricula or cultural contexts of other regions. This could put non-English speakers or those in different educational systems at a disadvantage.

Adapting this model to other high-resource languages like French or German would be relatively straightforward, requiring translation of the training datasets and subsequent fine-tuning. For low-resource languages like Swahili or Urdu, this process would be significantly more challenging due to the scarcity of translated STEM data.

Finally, the model could be misused for academic dishonesty. To mitigate this, it should be framed as a study aid for understanding processes, not just for obtaining answers. Although the reasoning process in MCQA is most of the time obfuscated, limiting understanding beyond a concise answer.

4 Conclusion

Our study demonstrates that building an efficient and specialized AI tutor for STEM tasks requires a multi-faceted approach. Supervised Fine-Tuning (SFT) provided solid performance gains, especially with our curated datasets. LoRA-based Parameter-Efficient Fine-Tuning (PEFT) proved particularly effective, outperforming full fine-tuning in most cases while being more lightweight and stable. Direct Preference Optimization (DPO) improved alignment with human preferences, especially when trained on high-quality, domain-specific data. Retrieval-Augmented Generation (RAG), while promising, showed limited impact due to noise and context length issues. Finally, post-training Quantization enabled efficient deployment with minimal performance degradation. Together, these techniques offer a practical and modular pipeline for enhancing compact models on specialized educational tasks. Future perspectives include refining retrieval, experimenting with larger models, and integrating user feedback.

References

[1] Intel orca dpo pairs. https://huggingface.co/datasets/Intel/orca_dpo_pairs. Accessed: 2025-05-29.

[2] Math-step-dpo-10k. https://huggingface.co/datasets/xinlai/Math-Step-DPO-10K. Accessed: 2025-05-29.

[3] Multifaceted-collection-dpo. https://huggingface.co/datasets/kaist-ai/Multifaceted-Collection-DPO. Accessed: 2025-05-29.

[4] Sciq dataset. https://huggingface.co/datasets/allenai/sciq. Accessed: 2025-05-29.

[5] Ultrafeedback binarized preferences. https://huggingface.co/datasets/argilla/ultrafeedback-binarized-preferences. Accessed: 2025-05-29.

[6] Aarushhh. mathsres dataset. https://huggingface.co/datasets/Aarushhh/math-reasoning-10k/viewer/default/train?row=0&views%5B%5D=train. Accessed: 2025-06-11.

[7] Asma Ben Abacha and Dina Demner-Fushman. 2019. A question-driven approach to building a corpus for medical question answering. In *Proceedings of the 18th BioNLP Workshop and Shared Task*, pages 33–42. Association for Computational Linguistics.

[8] Aida Amini, Saadia Hopson, Stefan Tworkowski, Aishwarya Kalyan, Swaroop Agrawal, Oleksandr Kuhn, Luis Lastras, Dhandapani Ravichandran, and Mrinmaya Sachan. 2019. MathQA: Towards interpretable math word problem solving with operation-based formalisms. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2357–2367, Minneapolis, Minnesota. Association for Computational Linguistics.

[9] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Llm.int8(): 8-bit matrix multiplication for transformers at scale. *arXiv preprint arXiv:2208.07339*.

[10] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms.

[11] Hugging Face. 2022. codeparrot/codeparrot-clean. https://huggingface.co/datasets/codeparrot/codeparrot-clean.

[12] Hugging Face. 2024. thewordsmiths/wiki-stem. https://huggingface.co/datasets/thewordsmiths/wiki-stem.

[13] Arnav Gudibande. 2023. Physics-alpaca dataset. https://huggingface.co/datasets/declare-lab/Physics-Alpaca.

[14] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*.

[15] Hamish Ivison, Ansong Wang, Eric Wallace, Sheng Li, Yizhong Li, Tom August, Jonathan Dodge, Alane Fan, Tianyu Gao, Jack Hessel, et al. 2023. Tülu 2: A family of instruction-tuned models for both research and commercial use. *arXiv preprint arXiv:2311.11070*.

[16] Daniel Khashabi, Sewon Min, Tushar Khot, Ashish Sabharwal, Oyvind Tafjord, Peter Clark, and Hannaneh Hajishirzi. 2020. Unifiedqa: Crossing format boundaries with a single qa system. *arXiv preprint arXiv:2005.00700*.

[17] Tushar Khot, Peter Clark, Michal Guerquin, Peter Jansen, and Ashish Sabharwal. 2020. QASC: A question answering dataset with sentence composition. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8081–8090.

[18] Nathan Lambert. 2023. preference-collection. https://github.com/CarperAI/preference-collection.

[19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.

[20] Pan Lu, Quang-Thang Le, and Nanyun Peng. 2022. ScienceQA: A new dataset for robust multimodal question answering. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8037–8053, Dublin, Ireland. Association for Computational Linguistics.

[21] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. 2018. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391, Brussels, Belgium. Association for Computational Linguistics.

[22] Zechen NLP. . Mnlp_dpo_eval dataset. https://huggingface.co/datasets/zechen-nlp/MNLP_dpo_eval. Accessed: 2025-06-11.

[23] Zechen NLP. . Mnlp_mcqa_eval dataset. https://huggingface.co/datasets/zechen-nlp/MNLP_mcqa_eval. Accessed: 2025-06-11.

[24] Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D Manning, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36.

[25] Alon Talmor, Jonathan Herzig, Nicholas Lourie, and Jonathan Berant. 2019. CommonsenseQA: A question answering challenge targeting common sense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4149–4158, Minneapolis, Minnesota. Association for Computational Linguistics.

[26] Johannes Welbl, Pontus Stenetorp, and Sebastian Riedel. 2018. Constructing datasets for multi-hop

reading comprehension across documents. In *Transactions of the Association for Computational Linguistics*, volume 6, pages 287–302, Cambridge, MA. MIT Press.

A Appendix

A.1 AI Usage Disclosure

Per the course AI policy, we report our use of AI-based tools in this project. We utilized different AI models, such as Gemini 2.5 Pro or GPT 4.1, for code debugging, refactoring, and report styling. In particular, AI assistants were used to help structure and draft sections of our progress and final reports based on our notes, data, and results. It helped ensure the language was clear, concise, and followed an academic style. We also used the assistants to help debug minor issues in our Python scripts for data preprocessing and model training.

In all cases, the AI's output was critically reviewed, verified, and edited by team members to ensure its correctness and relevance to our project. The core experimental design, model training, and analysis were conducted entirely by the project team. Data gathering for Milestone 1 was exempt from this requirement.

A.2 DPO part parameters

```
sft_params = {
    "learning_rate": 2e-4,
    "scheduler": "cosine",
    "batch_size": 3,
    "gradient_accumulation": 16,
    "epochs": 5
}

dpo_first_dataset_params = {
    "max_length": 768,
    "max_prompt_length": 384,
    "batch_size": 4,
    "gradient_accumulation": 16,
    "learning_rate": 2e-5,
    "epochs": 2,
    "beta": 0.05,
    "scheduler": "linear"
}

dpo_second_dataset_params = {
    "epochs": 1,
    #different, due to OOM problems
}
```

A.3 Additional quantization data

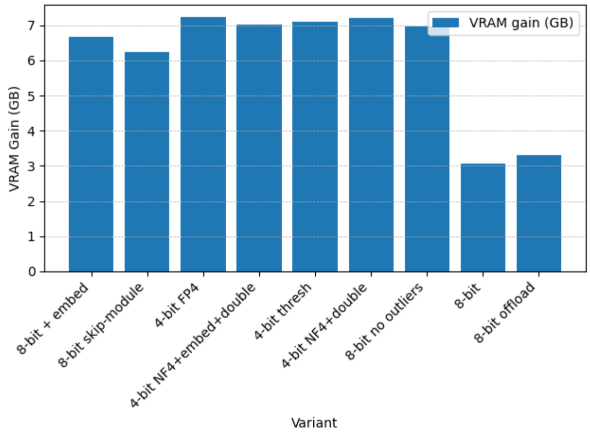

Figure 5: VRAM gain for each quantization variant relative to the full-precision baseline.

Table 6: Quantized models: Size, Compression Rate and Score

Variant	Size (MiB)	Compression rate	Score
8-bit + embed	733.41	1.57	16.93
8-bit skip	789.19	1.46	15.10
4-bit FP4	548.41	2.10	18.76
4-bit NF4+embed+double	529.07	2.18	18.17
4-bit thresh	548.41	2.10	18.04
4-bit NF4+double	529.07	2.18	19.37
8-bit no outliers	733.41	1.57	18.46
8-bit	733.41	1.57	8.31
8-bit offload	733.41	1.57	8.60
baseline FP	1152.06	1.00	5.81

Table 7: Quantized models: Accuracy, Accuracy Drop, VRAM peak and gains

Variant	Acc. (%)	Acc. drop (%)	Peak VRAM (GB)	VRAM gain (GB)
8-bit + embed	58.77	0.13	3.47	6.67
8-bit skip	58.88	0.02	3.90	6.24
4-bit FP4	54.80	4.10	2.92	7.22
4-bit NF4+embed+double	56.90	2.00	3.13	7.01
4-bit thresh	54.79	4.11	3.04	7.11
4-bit NF4+double	56.90	2.00	2.94	7.21
8-bit no outliers	58.62	0.28	3.18	6.97
8-bit	58.77	0.13	7.07	3.07
8-bit offload	58.77	0.13	6.83	3.31
baseline FP	58.90	0.00	10.14	0.00

A.4 RAG Encoder finetuning dataset

Pair Type	Question (Query)	Answer (Document)
Positive	What is the effect of dopant concentration on the critical temperature of a superconducting material?	In some cases, the addition of dopants can increase the critical temperature of the superconducting material by altering the electron pairing mechanism or the crystal lattice structure.
Negative	What is the Gibbs free energy change for the metabolic pathway of glycolysis?	Gravitational wave observations can be used to detect the presence of a population of primordial black holes (PBHs) in the universe by analyzing distortions in the observed wave signals.

Table 8: Illustrative examples of positive and negative (query, document) pairs used for contrastive learning. Positive pairs consist of a question and its corresponding answer, while negative pairs mix unrelated questions and answers.